SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR
Constituent of Symbiosis International (Deemed University), Pune
(Established under Section 3 of the UGC Act of 1956 wide notification number F-9-12/2001-U-3 of Government of India)
॥वसुधैव कुटुम्बकम्॥
Re-Accredited by NAAC with 'A++' Grade | Website: www.siu.edu.in | www.sitnagpur.edu.in
SYMBIOSIS Golden Jubilee
Celebrating 50 Years of Excellence

Course: Data Compression
Course Code: 705210502
Run Length Encoding

Dr. Gyana Ranjana Panigrahi

What is Run-Length Encoding (RLE)?

Definition:

Run-Length Encoding is a **lossless data compression technique** that stores sequences of the same data value (called "runs") as a **single value and count** instead of repeating the value multiple times.

Idea:

Instead of storing:
AAAAA

We store:
A5 (meaning 'A' repeated 5 times)

When is it useful?

RLE is most effective when:

- Data contains lots of repeating values.
- Example: Simple graphics, monochrome images, text with repeated characters, fax machine data, bitmap images with large uniform areas.

It’s not effective when:

- Data has little or no repetition (it can even make the file bigger).
- Example: Random binary data, encrypted files.

Example: 0101011001101010010110 10... (where each bit is independent and has a 50% chance of being 0 or 1).

How RLE Works – Step by Step

Let's encode the string:
WWWWWBBWWWWWW

Step 1 — Identify runs

- WWWWW → 4 W's
- BB → 2 B's
- WWWWWW → 6 W's

Step 2 — Store as (symbol, count) pairs

- (W,4), (B,2), (W,6)

Step 3 — Represent in compressed form

- Option 1 (Text format): W4B2W6
- Option 2 (Binary format): Store symbol and count in fixed bits

Algorithm

Encoding:

1. Initialize an empty output list.
2. Start from the first character.
3. Count how many times the current character repeats consecutively.
4. Append (character, count) to output.
5. Move to the next new character.
6. Repeat until end of input.

Decoding:

1. Read the character and its count.
2. Repeat the character count times.
3. Append to output.
4. Continue until all pairs are processed.

Example – Text

Input: AAAABBBCCDAA

Encoding:

A4 B3 C2 D1 A2

Output:

A4B3C2D1A2

Decoding:

A4 → AAAA
B3 → BBB
C2 → CC
D1 → D
A2 → AA
Final: AAAABBBCCDAA

Example – Binary (Image Data)

Suppose a **1-bit black & white image row** is:

000000111110000

- 0 repeated 6 times → (0,6)
- 1 repeated 5 times → (1,5)
- 0 repeated 4 times → (0,4)

Compressed form:

(0,6)(1,5)(0,4)

Advantages

- Simple to implement – both encoding and decoding are easy.
- Lossless – no data loss after decompression.
- Works well on data with many repeats (e.g., icons, simple drawings).

Disadvantages

- Inefficient for high-entropy data (files with no repetition).
- May increase file size for random data.
- Not suitable for complex images with high color variation.

APPLICATIONS

- TIFF, BMP, PCX image formats
- Fax machines (CCITT Group 3 standard)
- PDF compression
- Bitmap fonts
- Simple game textures

TIFF: Tagged Image File Format
BMP: Bit map
PCX: PiCture eXchange

Pseudocode

Encoding:

```
def rle_encode(data):
    encoded = []
    i = 0
    while i < len(data):
        count = 1
        while i + 1 < len(data) and data[i] == data[i+1]:
            i += 1
            count += 1
        encoded.append((data[i], count))
        i += 1
    return encoded
```

Decoding:

```
def rle_decode(encoded):
    decoded = ""
    for char, count in encoded:
        decoded += char * count
    return decoded
```

Thank You